



“Formando líderes para la construcción de un nuevo país en paz”

Informe de avance: pruebas de los componentes del balancín seguidor de línea

Proceso de verificación individual de cada parte antes de armar el robot

Maikoll E. Santander Sepúlveda

Ingeniería Mecatrónica, Universidad de Pamplona

168418: CONTROL INTELIGENTE II

Ing. Jeisson Harvey Martínez Flórez

09 de octubre del 2026

“Formando líderes para la construcción de un nuevo país en paz”

Universidad de Pamplona

Pamplona - Norte de Santander - Colombia

Tels: (+57) 3153429495 - 3160244475

www.unipamplona.edu.co

SC-CER96940





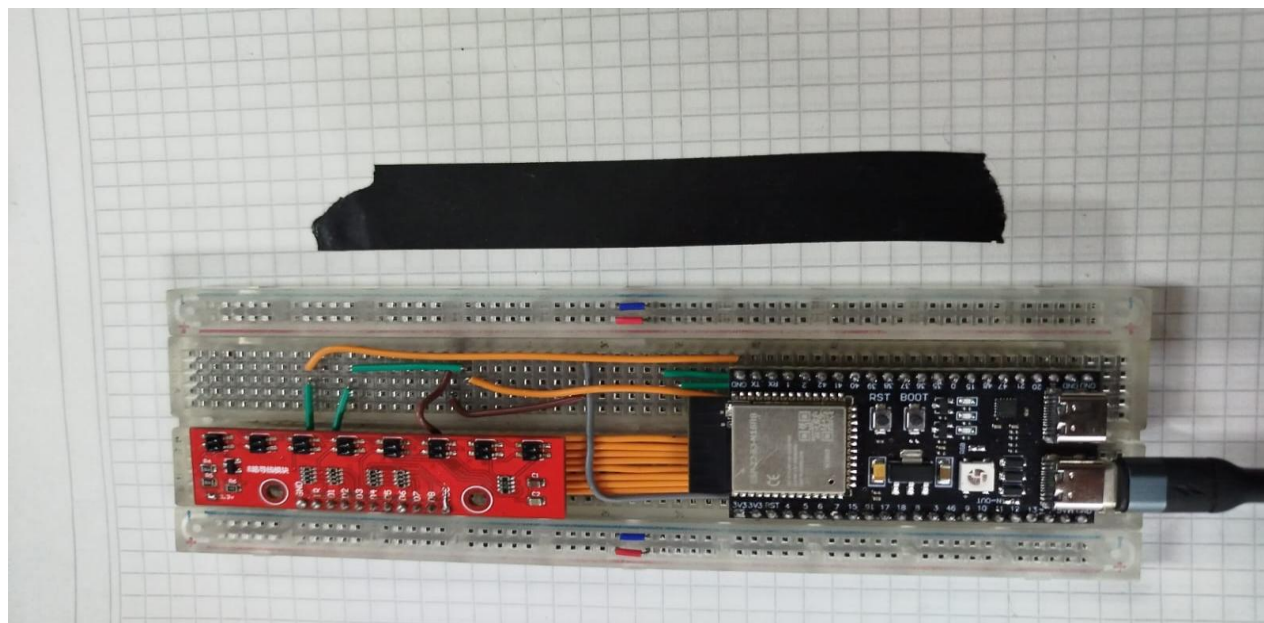
Con qué estoy trabajando

El proyecto es un balancín seguidor de línea, es decir, un robot de dos ruedas que tiene que mantenerse parado solo y además seguir una línea en el piso. Dentro del grupo me encargué de la parte de programación y de probar que cada componente funcionara.

Los componentes son: una ESP32-S3 (módulo N16R8) como controlador, un sensor de línea de ocho canales, un MPU6050 para medir la inclinación, dos motorreductores 25GA370 de 12 V y 100 RPM con encoder, llantas de 65 mm, y un puente H TB6612FNG. La idea es alimentarlo con una batería de litio de tres celdas, aunque para las pruebas usé una fuente de computador de 12 V porque todavía no teníamos las celdas.

Sensor de línea

Ilustración 1 Montaje y prueba de sensor de línea QTR



Fuente. Elaboración propia

Este fue el que más tiempo me quitó, y vale la pena contar el proceso completo porque casi todo lo que pasó fue por cosas que no se ven a simple vista.

Arranqué con una regleta que venden como digital. La revisé, no tenía potenciómetros ni comparadores, así que asumí que era del tipo RC, que son las que se leen midiendo cuánto tarda cada canal en descargarse: si la superficie es blanca refleja mucho y el tiempo es corto, si es negra el tiempo es largo. Monté el código con ese método y los ocho canales me daban siempre el mismo valor, el tope del tiempo de espera. Eso significaba que las salidas nunca bajaban.

Para salir de dudas la leí como analógica y ahí sí me cambiaron los valores, pero se quedaban pegados arriba, cerca de 4095, sin importar si la ponía sobre blanco o sobre negro. O sea que la placa sí era analógica y no RC como yo creía. Primera lección: lo que dice el vendedor no sirve para saber cómo



funciona realmente el sensor, toca comprobarlo.

Pero seguía sin servir. Probé acercándole el dedo a cada canal y solo reaccionaba cuando lo tocaba, y apenas un poquito. Eso apuntaba a que los emisores infrarrojos estaban iluminando muy débil.

Revisando la placa encontré unos pads marcados 3.3v que estaban abiertos: resulta que estas regletas vienen hechas para 5 V, y ese puente es el que hay que soldar para que los LEDs tengan el brillo correcto cuando uno las alimenta a 3,3 V, que es lo que maneja la ESP32. Lo soldé y mejoró algo, pero no lo suficiente para distinguir bien el blanco del negro.

En medio de esas pruebas me pasó otra cosa que me costó varias horas. Los valores me salían casi idénticos en todos los canales y además iban bajando solos poquito a poco, sin relación con lo que tenía debajo. Eso no tiene sentido en sensores reales, porque cada fototransistor varía un poco. Me puse a medir continuidad con el multímetro y encontré que uno de los jumpers de tierra no tenía continuidad, aunque estaba bien puesto en la protoboard. Lo cambié y al instante los valores se separaron y empezaron a responder a la luz.

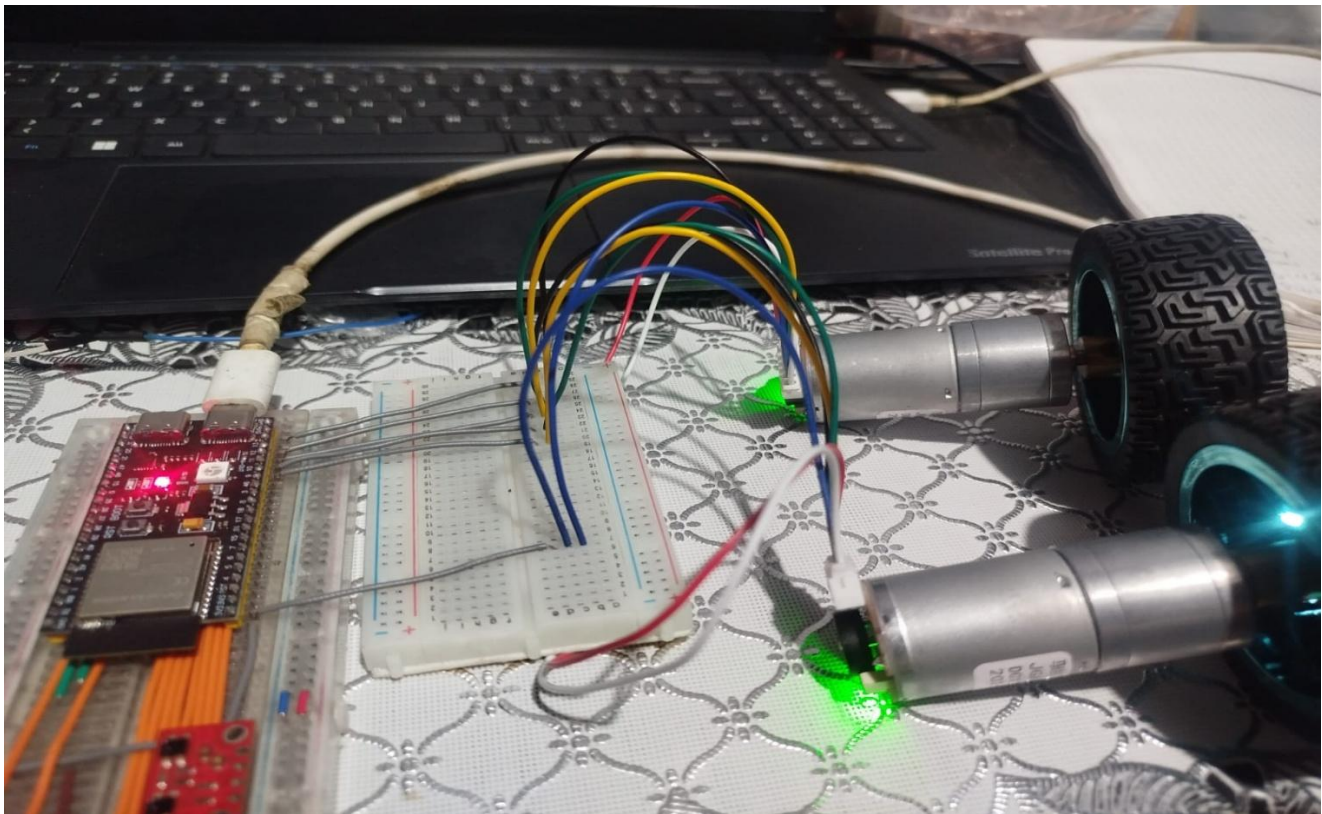
Al final conseguí prestada otra regleta, esta sí de tipo RC, y funcionó de una. Sobre blanco los canales me daban entre 0 y 243 microsegundos, y sobre la cinta negra todos se iban al tope de 2 500. Es una diferencia de más de diez veces, que es justo lo que uno necesita para seguir línea sin que la luz del salón lo afecte. Corrí también una rutina que registra el mínimo y el máximo de cada canal mientras uno pasa el sensor de un lado a otro, y los ocho dieron rangos entre 2 380 y 2 500, o sea que ninguno quedó malo.

Algo que aproveché fue el pin LEDON, que es el que enciende los emisores. Como en operación normal siempre va encendido, en vez de gastarme un pin del microcontrolador lo conecté directo a 3,3 V y así

me quedó un pin libre para otra cosa.

Encoders de los motores

Ilustración 1 Montaje y prueba de encoders



Fuente. Elaboración propia

Los motores traen encoder magnético de dos canales. El fabricante dice que dan once pulsos por vuelta del motor, pero como hay caja reductora de por medio, lo que a mí me interesa es cuántos pulsos hay por vuelta de la llanta, y eso tocaba medirlo.



El código que hice usa una interrupción en el flanco de subida del canal A, y dentro de la interrupción revisa el canal B para saber hacia dónde va girando: si el canal B está en alto suma, si no, resta. Así el contador sube o baja según el sentido.

Al principio intenté girar el eje con unos alicates y no pude, porque la caja reductora no se deja mover desde la salida. Monté las llantas y ahí sí quedó fácil. La primera medición la hice con una sola vuelta y me dio 800 en un motor y 849 en el otro, una diferencia del 6 % que me preocupó. Repetí con diez vueltas y me dio 8 224 y 8 227, o sea 822,4 y 822,7 pulsos por vuelta. La diferencia real entre motores es del 0,04 %, nada. Lo de antes había sido error mío al parar la llanta en la marca.

Con ese número también se puede sacar la reducción: 822,4 dividido entre los once pulsos del motor da cerca de 74,8, que coincide con una relación de 1:75, que es la típica de estos motores. Y como la llanta de 65 mm tiene unos 204 mm de circunferencia, cada pulso equivale a más o menos 0,25 mm de recorrido, que es bastante buena resolución para el control de velocidad. También comprobé que al girar al revés el contador baja, que era lo que faltaba verificar.

Consumo de los motores

Antes de conectar el puente H quería saber cuánta corriente piden los motores, porque de eso depende si el driver aguanta. La ficha de la tienda no trae ese dato, solo voltaje, velocidad y torque, así que tocó medirlo.

Conecté el multímetro en serie con el motor, en corriente continua y escala de 10 A, alimentando con la fuente de computador a 12 V. En vacío me dio 0,5 A, y frenando la llanta con la mano un segundo me



dio 1,5 A. Ese segundo valor es el importante, porque el TB6612FNG maneja alrededor de 1 A continuo y 3 A de pico. O sea que estoy por encima del continuo, pero por debajo del pico.

En la práctica creo que va a funcionar, porque en un balancín los motores hacen correcciones rápidas y casi nunca quedan bloqueados, pero igual queda ahí un margen justo.

Puente H TB6612FNG

Ilustración 1 Montaje y prueba de puente H TB6612FNG





Fuente. Elaboración propia

Para esta prueba hice un programa con comandos por el monitor serial: una tecla para cada motor hacia adelante, otra hacia atrás, una para parar todo y otras dos para subir y bajar la velocidad.

Ya compilando, conecté todo y los motores no se movían. El monitor sí me confirmaba que recibía los comandos, así que empecé a descartar. Medí la alimentación del módulo y estaba bien, tanto la de potencia como la lógica, y el pin STBY también estaba en alto, que es el que habilita el driver. Después medí las señales directamente en los pines de la ESP32 y también estaban correctas: la dirección bien y el PWM con el voltaje promedio que correspondía. Entonces medí entre las salidas que van al motor y no había nada, cero voltios. Ahí concluí que el módulo estaba dañado.

Cambié el módulo por otro que tenía a la mano y funcionó de inmediato, lo que confirmó el diagnóstico.

Los dos canales responden bien en ambos sentidos y la velocidad cambia proporcional al PWM.

Además, probé hasta dónde baja y los motores siguen girando con un PWM de 20 sobre 255, como un 8 % de la escala. Eso es bueno para el control, porque significa que las correcciones pequeñas sí se traducen en movimiento. Claro que esa medición fue sin carga; con el peso del robot encima el mínimo va a subir.

MPU6050

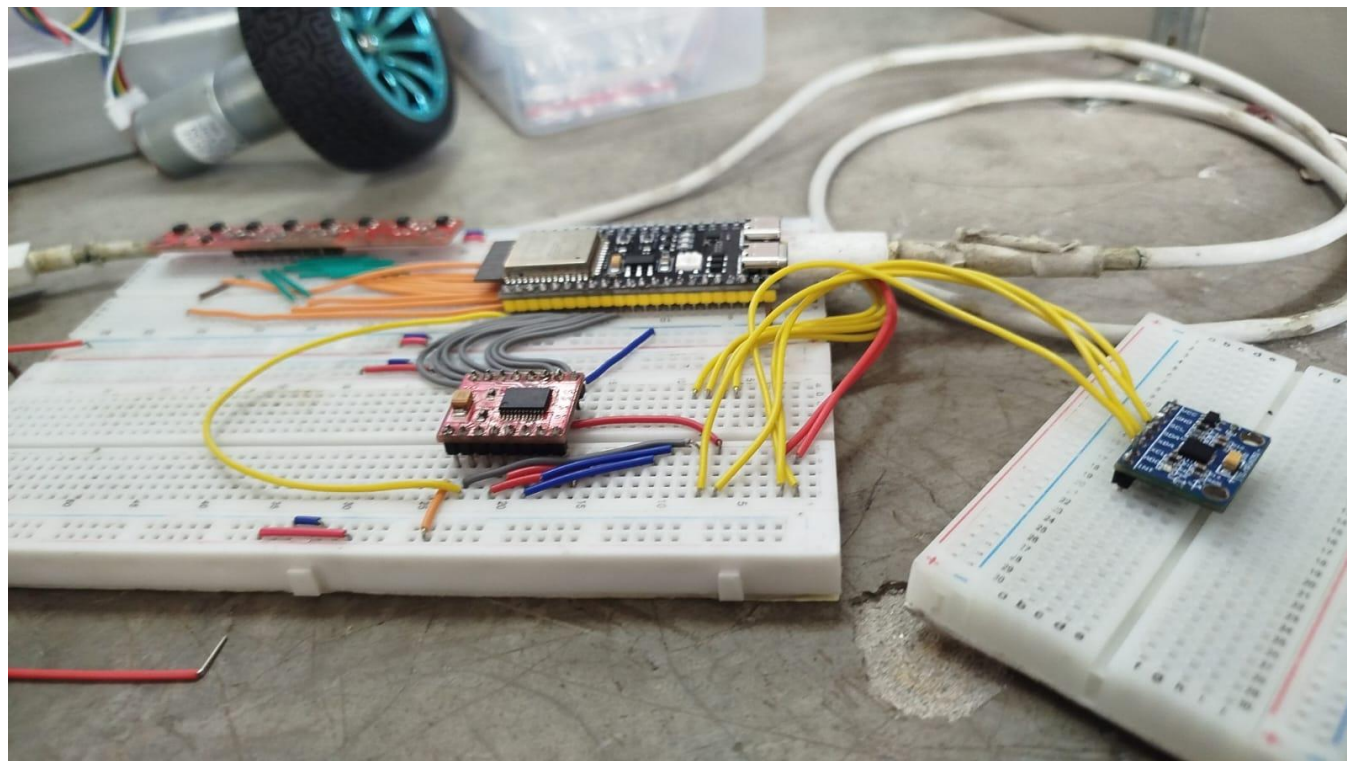


"Formando líderes para la construcción de un nuevo país en paz"

Universidad de Pamplona
Pamplona - Norte de Santander - Colombia
Tels: (+57) 3153429495 - 3160244475
www.unipamplona.edu.co



Ilustración 1 Montaje y prueba de Sensor Giroscopio Acelerómetro MPU6050



Fuente. Elaboración propia

Este es el sensor más importante para el equilibrio, porque es el que le dice al control qué tan inclinado va el robot. Monté el código, me conectó bien por I²C y me daba valores que cambiaban al moverlo, pero no eran los que yo esperaba: si lo inclinaba 90 grados no me marcaba 90 ni nada parecido.

Revisando entendí el error. El MPU6050 no entrega ángulo. El acelerómetro entrega aceleración en metros por segundo al cuadrado, y de ahí uno deduce la inclinación viendo cómo se reparte la gravedad entre los ejes; y el giroscopio entrega velocidad angular, que cuando el sensor está quieto marca casi cero sin importar cómo esté inclinado. Ninguno de los dos dice directamente la orientación, por eso yo



no veía lo que buscaba.

Entonces implementé un filtro complementario, que es la forma común de resolverlo. La idea es combinar las dos señales aprovechando lo bueno de cada una: del acelerómetro se saca un ángulo absoluto que no se va con el tiempo pero que es muy ruidoso, y del giroscopio se saca un ángulo integrando la velocidad, que es suave y rápido pero que se va desviando poco a poco. El filtro le da más peso al giroscopio en el corto plazo y usa el acelerómetro para corregir la deriva. Lo dejé en 0,98 de arranque. También le agregué una rutina que promedia 500 muestras con el sensor quieto para medir la desviación del giroscopio en cero y restarla después.

Para verificarlo dejé que el programa mostrara las tres estimaciones al tiempo y la diferencia se ve clarísima: la del acelerómetro salta mucho, la del giroscopio es suave pero se va corriendo sola si uno deja el sensor quieto, y la del filtro se queda estable. Al inclinarlo 90 grados en el sentido en que quedaría montado en el robot, el filtro marca un valor cercano a 90 y se mantiene ahí. También confirmé que el eje que estoy usando es el de inclinación hacia adelante y atrás, que es el que importa para el equilibrio.

Cómo quedaron los pines

Cuando mi compañero tuvo el esquemático le revisé la asignación de pines y encontré dos problemas. El bus I²C del MPU estaba en los GPIO 19 y 20, que en la ESP32-S3 son el USB nativo, y como yo cargo los códigos por ese puerto no podía dejarlos ahí. El otro era un LED en el GPIO 37, que en el módulo N16R8 está ocupado por la memoria PSRAM. Los dos se movieron a tiempo, antes de mandar a fabricar la tarjeta, y actualicé todos los programas de prueba con la asignación definitiva.



Tabla 1

Asignación final de pines

Componente	Señal	GPIO
Sensor de línea	D1 a D8	18, 8, 9, 10, 11, 12, 13, 14
Sensor de línea	LEDON	Directo a 3,3 V
Puente H	PWMA, AIN2, AIN1, STBY	4, 5, 6, 7
Puente H	BIN1, BIN2, PWMB	15, 16, 17
Encoder motor A	Canales A y B	1, 2
Encoder motor B	Canales A y B	42, 41
MPU6050	SDA, SCL, INT	48, 47, 21
Pulsadores	Calibrar e iniciar	40, 39
LED indicador	—	45

En qué voy y qué sigue

Todos los componentes quedaron probados y funcionando: el sensor de línea distingue perfectamente blanco de negro en sus ocho canales, los encoders dan 822 pulsos por vuelta con detección de sentido y los dos motores salieron prácticamente idénticos, el consumo de los motores es manejable para el driver, el puente H mueve los dos motores en ambos sentidos con buen control de velocidad, y el MPU6050 ya entrega un ángulo estable con el filtro complementario.

Lo que sigue es el lazo de equilibrio sobre el chasis ya armado, usando solo el MPU6050 y los motores, sin la línea todavía. La idea es ir por partes: primero que se mantenga parado en el sitio, después que se



mantenga parado moviéndose, luego que pueda girar sin caerse, y de último meterle el seguimiento de línea encima. Si trato de ajustar el equilibrio y la línea al mismo tiempo no voy a saber cuál de los dos me está generando las oscilaciones.



"Formando líderes para la construcción de un nuevo país en paz"

Universidad de Pamplona
Pamplona - Norte de Santander - Colombia
Tels: (+57) 3153429495 - 3160244475
www.unipamplona.edu.co